

DESIGN & ANALYSIS OF ALGORITHM

(FALL 2024)

Solution by (Tanzeel Zahra)

Short Questions

1) At most how many edges can be drawn in an undirected graph with k vertices (no self-loops)?

1. In an undirected graph, an edge connects two distinct vertices.
2. The number of ways to choose 2 vertices from k vertices is:

$$\binom{k}{2} = \frac{k(k-1)}{2}$$

3. Therefore, the maximum number of edges is:

$$\frac{k(k-1)}{2}$$

Explanation:

- Each edge is counted once.
- No self-loops are allowed.
- A graph with this many edges is called a complete graph.

Example:

For $k=5$ vertices:

$$\frac{5 \cdot 4}{2} = 10 \text{ edges}$$

2) What is a minimum spanning tree of a graph? In which case two different MSTs may have the same cost?

MST Definition: A Minimum Spanning Tree of a connected, weighted undirected graph is a tree that spans all vertices with the minimum total edge weight.

Properties:

- Has exactly $V-1$ edges if V is the number of vertices.
- No cycles.
- May not be unique if multiple edges have the same weights.

Same cost MSTs Example:

Consider a square graph with vertices A, B, C, D and edges:

$AB = 1, BC = 2, CD = 1, DA = 2, AC = 2, BD = 2$

Two possible MSTs:

$\{AB, BC, CD\} \rightarrow \text{total} = 1+2+1 = 4$

$\{AB, AD, CD\} \rightarrow \text{total} = 1+2+1 = 4$

Reason: Equal-weight edges allow different tree structures with same total weight.

General Case:

When all edges have the same weight, every spanning tree is an MST with same cost = $(V-1) \times w$

3) Running time of BFS with adjacency matrix representation

1. Graph Representation:

- Adjacency Matrix of size $V \times V$, where V = number of vertices.

2. BFS Algorithm:

- BFS visits each vertex once using a queue.
- For each vertex, it checks all other vertices in its adjacency matrix row to find neighbors.

3. Time Complexity Computation:

For V vertices, checking V entries each $\rightarrow V \times V = V^2$.

$$\text{Time Complexity} = O(V^2)$$

4. Space Complexity:

- Adjacency matrix: $O(V^2)$
 - Queue + visited array: $O(V)$
-

4) Solve recurrence relations

a) $T(n)=4T(n/2)+n$

Using Master Theorem

$$T(n) = aT\left(\frac{n}{b}\right) + f(n)$$

Step 1: Identify parameters

- $a = 4$
- $b = 2$
- $f(n) = n$

Step 2: Compute $n^{\log_b a}$

$$\log_b a = \log_2 4 = 2$$

$$n^{\log_b a} = n^2$$

Step 3: Compare $f(n)$ with $n^{\log_b a}$

- $f(n) = n$
- $n^{\log_b a} = n^2$

Since:

$$f(n) = O(n^{2-1}) = O(n)$$

This matches Case 1 of the Master Theorem.

Step 4: Apply Case 1

Case 1 formula:

$$T(n) = \Theta(n^{\log_b a})$$

$$T(n) = \Theta(n^2)$$

Final Answer:

$$T(n) = \Theta(n^2)$$

b) $T(n)=5T(n/2)+n^2$

$$T(n) = aT\left(\frac{n}{b}\right) + f(n)$$

Step 1: Identify parameters

- $a = 5$
- $b = 2$
- $f(n) = n^2$

Step 2: Compute $n^{\log_b a}$

$$\log_b a = \log_2 5 \approx 2.32$$

$$n^{\log_b a} \approx n^{2.32}$$

Step 3: Compare $f(n)$ with $n^{\log_b a}$

- $f(n) = n^2$
- $n^{\log_b a} \approx n^{2.32}$

Since:

$$f(n) = O(n^{2.32-\varepsilon}) \quad \text{for some } \varepsilon > 0$$

This satisfies **Case 1** of the Master Theorem.

Step 4: Apply Case 1

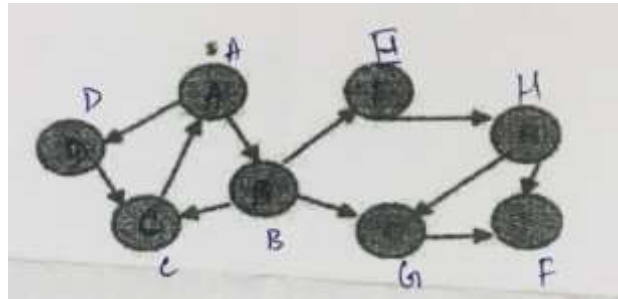
Case 1 formula:

$$T(n) = \Theta(n^{\log_b a})$$

Final Answer:

$$T(n) = \Theta(n^{\log_2 5}) \approx \Theta(n^{2.32})$$

5) DFS traversal order from "A"



DFS Rule:

DFS starts from the given node, visits nodes as deep as possible, and backtracks when no unvisited neighbor is left.

DFS starting node: A

DFS Traversal Order

$A \rightarrow D \rightarrow C \rightarrow B \rightarrow E \rightarrow H \rightarrow G \rightarrow F$

DFS Start Time & End Time

Node	Start Time	End Time
A	1	16
D	2	5
C	3	4
B	6	15
E	7	14
H	8	13
G	9	12
F	10	11

6) Heap sort and its time complexity

Heap Sort is a comparison-based sorting algorithm that uses a Binary Heap data structure. In Heap Sort, the array is first converted into a Max Heap, where the largest element is at the root.

The root element is repeatedly removed and placed at the end of the array, and the heap property is restored using heapify.

Algorithm Steps

1. Build a Max Heap from the given array.
2. Swap the root (largest element) with the last element.
3. Reduce heap size by one.
4. Apply heapify to maintain heap property.
5. Repeat until the array is sorted.

Time Complexity

- **Best Case:** $O(n \log n)$
- **Average Case:** $O(n \log n)$
- **Worst Case:** $O(n \log n)$

Space Complexity

- **$O(1)$** (In-place sorting)

Long Questions

1) Pseudo code using DFS to find all components in undirected graph

Pseudo Code:

```
function findAllComponents(graph):
    visited = set()           // To track visited vertices
    components = []          // List to store each component (as list of vertices)

    for each vertex v in graph.vertices:
        if v not in visited:
            component = []    // Store vertices in current component
            DFS(v, visited, component, graph)
            components.append(component)

    return components

function DFS(v, visited, component, graph):
    visited.add(v)
    component.append(v)

    for each neighbor u of v in graph.adjacencyList:
        if u not in visited:
            DFS(u, visited, component, graph)
```

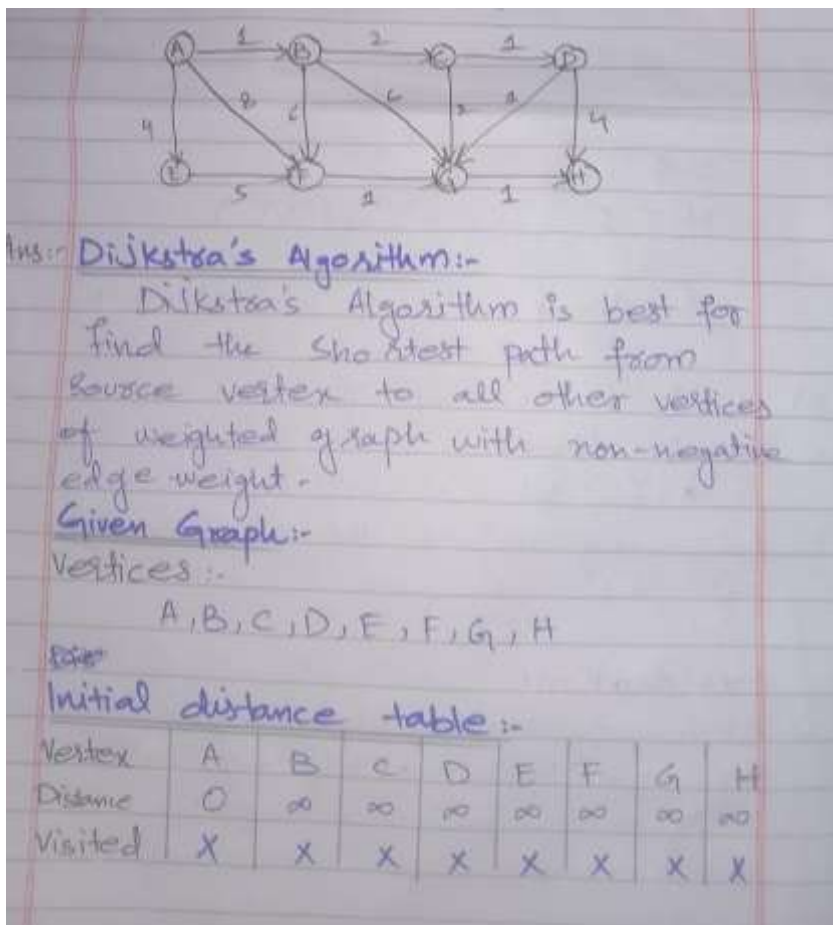
Explanation:

1. We iterate over all vertices.
2. If a vertex is not visited, start a new component and run DFS from it.
3. During DFS, mark all reachable vertices as visited and add them to the current component.
4. After DFS finishes, store the component in `components` list.
5. Repeat until all vertices are visited.

Complexity:

- **Time:** $O(V+E)$ where V is vertices, E is edges.
- **Space:** $O(V)$ for visited set and recursion stack.

2) Dijkstra's algorithm from vertex A:



Step 1:-
Select A (distance = 0 → minimum)

Vertex	A	B	C	D	E	F	G	H
distance	0	1	2	4	4	7	5	6
Visited	✓	X	X	X	X	X	X	X

Step 2:-
Select B (distance = 1 → minimum)

Vertex	A	B	C	D	E	F	G	H
distance	0	1	2	4	4	7	5	6
Visited	✓	✓	X	X	X	X	X	X

Step 3:-
Select C (distance = 2)

Vertex	A	B	C	D	E	F	G	H
distance	0	1	2	4	4	7	5	6
Visited	✓	✓	✓	X	X	X	X	X

Step 4:-
Select D (distance = 4)

Vertex	A	B	C	D	E	F	G	H
distance	0	1	2	4	4	7	5	6
Visited	✓	✓	✓	✓	X	X	X	X

Step 5:- select E

Vertex	A	B	C	D	E	F	G	H
Distance	0	1	2	4	4	7	5	6
Visited	✓	✓	✓	✓	✓	X	X	X

Step 6:-
Select G (distance = 5)

Vertex	A	B	C	D	E	F	G	H
Distance	0	1	2	4	4	7	5	6
Visited	✓	✓	✓	✓	✓	X	✓	X

Step 7:-
Select H (distance = 6)

Vertex	A	B	C	D	E	F	G	H
Distance	0	1	2	4	4	7	5	6
Visited	✓	✓	✓	✓	✓	X	✓	✓

Step 8:-
Select F (distance = 7)

Vertex	A	B	C	D	E	F	G	H
Distance	0	1	2	4	4	7	5	6
Visited	✓	✓	✓	✓	✓	✓	✓	✓

Final Table :-

Vertex	Distance From A
A	0
B	1
C	2
D	4
E	4
F	7
G	5
H	6

3) C++ function for pivot in QuickSort

```
#include <iostream>
#include <algorithm>
using namespace std;

int partition(int arr[], int low, int high) {
    // Choosing last element as pivot
    int pivot = arr[high];
    // Index of smaller element
    int i = low - 1;

    for (int j = low; j < high; j++) {
        // If current element is smaller than pivot
        if (arr[j] <= pivot) {
            i++;
        }
    }
}
```

```

        swap(arr[i], arr[j]);
    }
}
// Place pivot in correct position
swap(arr[i + 1], arr[high]);
return i + 1; // Return pivot index}

int main() {
int arr[] = {10, 80, 30, 90, 40, 50, 70};
int n = sizeof(arr)/sizeof(arr[0]);
int pivotIndex = partition(arr, 0, n-1);
cout << "Pivot position: " << pivotIndex << endl;
return 0;
}

```

Explanation:

- Pivot selection here is last element (Lomuto partition scheme).
- All elements less than pivot go to left, greater to right.
- Returns final index of pivot after partitioning.
- Time: $O(n)$ for partition step.